

MİKROİŞLEMCİLER DERSİ

DÖNEM PROJESİ RAPORU

ASCII TANK OYUNU

8086 Assembly Dili ile Geliştirilmiş

emu8086 Emülatöründe Çalıştırılmak Üzere Tasarlanmıştır

Proje Adı	ASCII Tank Oyunu
Programlama Dili	8086 Assembly (COM formatı)
Çalışma Ortamı	emu8086 Emülatörü
Ders	Mikroişlemciler
Tarih	05.05.2026

1. Giriş ve Proje Amacı

Bu proje, 8086 Assembly dili kullanılarak geliştirilen, emu8086 emülatöründe çalışan bir ASCII tank oyunudur. Program COM formatında yazılmış olup DOS ortamında doğrudan çalıştırılabilir.

Oyunun amacı; oyuncunun 20 mermiyle 8 hedefi vurmasıdır. Hedeflerin tamamı vurulduğunda oyun kazanılır, mermiler bittiğinde ise kaybedilir. En iyi skor TANK.DAT dosyasına kaydedilerek kalıcı hale getirilir.

1.1 Oyun Mekanikleri

- Oyuncu tankı ok tuşları veya WASD ile hareket ettirir
- SPACE tuşuyla ateş edilir, ESC ile oyundan çıkılır
- Mermiler engellere çarptığında yok olur, hedeflere çarptığında 10 puan kazanılır
- Oyun alanı sınırları: satır 3–22, sütun 1–78
- Maksimum 20 eş zamanlı mermi desteklenir

2. Program Yapısı ve Akışı

Program COM formatında tek bir segment üzerinde çalışır. ORG 100h direktifi ile başlayan kod, ana döngüden itibaren menü → oyun → oyun sonu → menü şeklinde bir akış izler.

2.1 Ana Program Akışı

```
main:
    CALL load_best_score      ; Dosyadan en iyi skoru yükle
mm_loop:
    CALL show_menu           ; Menüü göster
    CALL wait_menu_input     ; ENTER veya ESC bekle
    CALL init_game           ; Oyunu sıfırla
    CALL play_game           ; Oyun döngüsü
    CALL update_best_score   ; Skor güncelle
    CALL save_best_score     ; Dosyaya kaydet
    CALL show_gameover       ; Sonuç ekranı
    JMP mm_loop              ; Başa dön
```

2.2 Temel Fonksiyonlar

Fonksiyon	Görevi
put_char	Video belleğine (0B800h) tek karakter yazar
draw_string	Null-terminated string ekrana yazar
do_move	Tank hareketini işler; sınır ve engel kontrolü yapar
do_fire	Boş mermi slotu bulur ve mermiyi başlatır
update_bullets	Her tick'te aktif mermileri bir adım hareket ettirir
bullet_collide_check	Mermi çarpışma kontrolü: sınır, engel, hedef
is_obstacle	Koordinatın engel olup olmadığını kontrol eder
load/save_best_score	TANK.DAT dosyasından skor okuma/yazma

3. Ekran Yönetimi

Ekrana yazma işlemleri için BIOS kesmesi (INT 10h) yerine doğrudan video belleğine erişim tercih edilmiştir. Bu yaklaşım, özellikle sürekli güncellenen oyun elemanlarında belirgin bir hız avantajı sağlar.

3.1 Video Belleğine Doğrudan Erişim

80x25 metin modunda ekran, 0B800h segment adresinden başlayan bir bellek bölgesine karşılık gelir. Her karakter 2 byte yer kaplar:

- 1. byte: ASCII karakter kodu
- 2. byte: Renk niteliği (üst nibble arka plan, alt nibble ön plan rengi)

```
; put_char fonksiyonundan ilgili kısım:
MOV AX, VIDEO_SEG      ; VIDEO_SEG EQU 0B800h
MOV ES, AX
; Offset = row_offsets[satir] + sutun*2
XOR BX, BX
MOV BL, DH              ; DH = satir
SHL BX, 1               ; Her satir 2 byte: row_offsets indeksi
MOV DI, [row_offsets + BX]
```

```
MOV AL, CH      ; CH = karakter
MOV AH, CL      ; CL = renk
MOV ES:[DI], AX ; Belleğe yaz
```

3.2 Renk Nitelikleri

Programda kullanılan renk sabitleri şunlardır:

Sabit	Değer	Kullanım Yeri
ATTR_TANK	0Eh	Tank karakteri (sarı)
ATTR_BULLET	0Fh	Mermi (parlak beyaz)
ATTR_TARGET	0Ch	Hedef (kırmızı)
ATTR_OBSTACLE	08h	Engeller (koyu gri)
ATTR_BORDER	09h	Oyun alanı çerçevesi (mavi)
ATTR_STATUS	0Ah	Durum çubuğu (yeşil)

4. Kesme (Interrupt) Kullanımı

Program üç farklı yazılım kesmesini kullanmaktadır. Her kesme, farklı bir donanım servisine erişim sağlar.

4.1 Kullanılan Kesmeler

Kesme	AH Değeri	İşlev
INT 10h	AX=0003h	80x25 metin modunu etkinleştir
INT 10h	AH=01h	İmleci gizle/göster
INT 10h	AH=06h	Ekranı temizle (scroll up)
INT 16h	AH=00h	Klavyeden tuş oku (bekleyerek)
INT 16h	AH=01h	Tamponda tuş var mı? (beklemeden)
INT 21h	AH=3Ch	Dosya oluştur (creat)
INT 21h	AH=3Dh	Dosya aç (open)
INT 21h	AH=3Fh	Dosyadan oku (read)
INT 21h	AH=40h	Dosyaya yaz (write)
INT 21h	AH=3Eh	Dosyayı kapat (close)
INT 21h	AH=4Ch	Programı sonlandır (exit)

4.2 Non-Blocking Klavye Okuma

Oyun döngüsünün donmaması için INT 16h'nin AH=01h servisi kullanılır. Bu servis, Zero Flag'i kontrol ederek tamponda tuş olup olmadığını beklemeden sorgular:

```
; play_game fonksiyonundan:
pg_loop:
    MOV AH, 01h      ; Non-blocking sorgu
    INT 16h
```

```

JZ pg_tick      ; Tuş yoksa -> mermi güncellemesine git
MOV AH, 00h     ; Tuş varsa -> oku
INT 16h
CALL map_key    ; Tuşu oyun komutuna çevir

```

4.3 Dosya İşlemleri

En iyi skor, DOS INT 21h servisleri aracılığıyla TANK.DAT dosyasına kaydedilir ve program başladığında tekrar yüklenir:

```

; load_best_score fonksiyonundan:
MOV AH, 3Dh      ; Dosya aç
MOV AL, 0        ; Salt okunur
MOV DX, OFFSET score_file ; 'TANK.DAT'
INT 21h
JC lbs_none      ; Hata varsa (dosya yoksa) atla
MOV BX, AX       ; Dosya handle'ı BX'e
MOV AH, 3Fh      ; Oku
MOV CX, 2        ; 2 byte (WORD)
MOV DX, OFFSET best_score
INT 21h
MOV AH, 3Eh      ; Kapat
INT 21h

```

5. Veri Yapıları

5.1 Mermi Sistemi

Her mermi beş ayrı byte dizisinde tutulur. Bu yapı, sabit boyutlu bir slot tablosu oluşturarak dinamik bellek tahsisine gerek duymadan 20 eş zamanlı mermiyi yönetir:

```

bullet_active DB MAX_BULLETS DUP(0) ; Aktif mi? (0/1)
bullet_x      DB MAX_BULLETS DUP(0) ; X konumu
bullet_y      DB MAX_BULLETS DUP(0) ; Y konumu
bullet_dx     DB MAX_BULLETS DUP(0) ; X yönü (-1, 0, 1)
bullet_dy     DB MAX_BULLETS DUP(0) ; Y yönü (-1, 0, 1)

```

5.2 Engel Listesi

Engeller, koordinat çiftleri şeklinde ardışık byte'larla saklanır ve 0FFh sentinel değeriyle sonlanır. Bu yaklaşım, sabit uzunluklu dizi kullanmaktan daha esnektir:

```

obstacle_data:
    DB 10,6, 11,6, 12,6, 13,6, 14,6 ; x,y çiftleri
    DB 10,7, 10,8
    DB 30,9, 31,9, 32,9, ...
    DB 0FFh ; Liste sonu
; is_obstacle: 0FFh'e kadar doğrusal tarama
io_lp:
    MOV AL, [SI]
    CMP AL, 0FFh ; Sentinel kontrolü
    JE io_no     ; Eşleşme yok
    CMP AL, DL   ; X kontrolü
    JNE io_skip
    MOV AL, [SI + 1]
    CMP AL, DH   ; Y kontrolü
    JNE io_skip

```

```
MOV AL, 1          ; Engel bulundu
JMP io_end
```

6. Sonuç

Bu proje, 8086 Assembly dilinin temel kavramlarını — kesme servisleri, doğrudan bellek erişimi, yazmaç yönetimi ve düşük seviyeli I/O — pratik bir uygulama üzerinde bir araya getirmektedir.

Projenin öne çıkan teknik özellikleri şunlardır:

- BIOS kesmesi yerine doğrudan video belleği erişimiyle hızlı ekran güncelleme
- Non-blocking klavye okuma ile akıcı oyun döngüsü
- Sentinel değer tabanlı engel listesi ile esnek veri yapısı
- DOS INT 21h servisleri ile kalıcı skor kaydı
- 20 slotluk mermi tablosu ile eş zamanlı mermi yönetimi